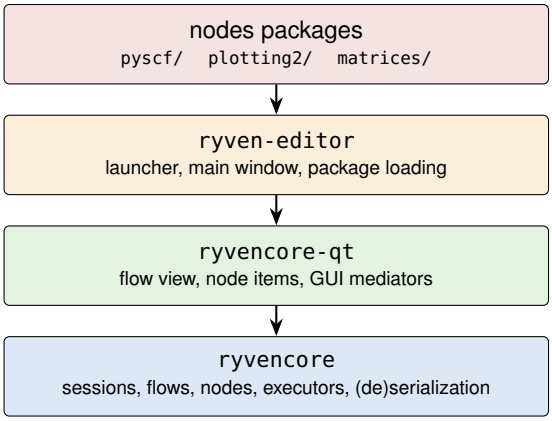
How Ryven Works Internally

A tour of ryvencore, ryvencore-qt, and the Ryven editor

— written for the Visual-SCF project —



Based on the checkouts at `~/Projects/Ryven` (fork of Ryven 3.5) and `~/Projects/ryvencore` (v0.5.0), as installed editable into Visual-SCF's venv.

July 2026

Contents

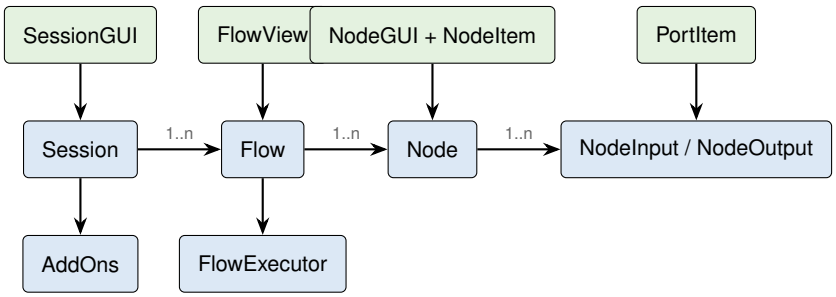
1 The big picture

Ryven is not one program but a stack of three Python packages, deliberately split so that the graph engine has no GUI dependency at all:

- **ryvencore** — the headless flow engine. It knows about sessions, flows (graphs), nodes, ports, connections, data wrappers, execution algorithms, add-ons, and how to serialize all of it to a JSON-compatible dict. It never imports Qt. Lives in its own repository (`~/Projects/ryvencore`).
- **ryvencore-qt** — the Qt frontend for ryvencore. It wraps a core `Session` in a `SessionGUI`, renders each `Flow` as a `QGraphicsScene`-based `FlowView`, and mediates between abstract nodes and their visual items. Lives in the Ryven monorepo (`Ryven/ryvencore-qt`).
- **ryven-editor** — the application: command-line parsing, the startup dialog, the main window, the nodes-package import system (`ryven.node_env` / `ryven.gui_env`), themes, the built-in console, and the code editor. Lives in `Ryven/ryven-editor`.

A *nodes package* such as Visual-SCF's `pyscf/` sits on top of all three: its `nodes.py` imports only from `ryven.node_env` (which re-exports ryvencore types), while its `gui.py` imports from `ryven.gui_env` (which re-exports ryvencore-qt widget classes). This split is what makes node logic importable in headless contexts.

The key mental model for everything that follows:



The bottom row (blue) is pure ryvencore and works without a display. The top row (green) is ryvencore-qt: each GUI object holds a reference to the core object it represents, subscribes to its events, and adds a Qt item to the scene. The core never calls into the GUI directly — it emits events; whether anything listens is the frontend's business.

2 ryvencore: the object model

2.1 `Base`: identity, events, and the serialization hook

Almost every core class (`Session`, `Flow`, `Node`, `NodePort`, `Data`) derives from `Base` (`ryvencore/Base.py`), which provides three things:

Global IDs. Every object gets a `global_id` from a process-wide counter. When an object is loaded from a save file, its old ID is remembered as `prev_global_id` and registered in a static map, so components that stored references by ID (notably GUI state) can re-associate after load. (The map is never pruned — a known, commented, benign leak.)

A tiny pub/sub system. `Event` is a minimal observer implementation: `sub(callback, nice=0)`, `unsub()`, `emit(*args)`. The `nice` value orders callbacks (lower fires first); internal wiring such as flow bookkeeping subscribes at `nice=-5` so it runs before user-visible handlers. This is the mechanism behind `flow.node_added`, `node.updating`, `node.update_error`, etc. — and ryvencore-qt's whole job is translating these into Qt signals.

The `complete_data` hook. Serialization is initiated by the core (`data()` methods returning dicts), but the frontend can register a single static function, `Base.complete_data_function`, which is applied to serialized dicts to *supplement* them with GUI-only state. This is how node positions and widget state end up inside a project file without the core knowing they exist (section 6).

2.2 `Session`

`Session` (`ryvencore/Session.py`) is the project root. It owns:

- `flows` — the list of `Flow` objects (a Ryven project can have several tabs, each is a flow);
- `nodes` — a *set of node classes* registered via `register_node_type()`; only registered classes may be instantiated in flows, and loading a project resolves node identifiers against this set;
- `data_types` — registered `Data` subclasses, keyed by identifier (needed to reconstruct typed payloads on load);
- `addons` — add-on singletons discovered from `ryvencore/addons/*.py` (Variables, Logging) when constructed with `load_addons=True`.

`Session.load(data)` restores add-ons first, then recreates each flow. It contains backward-compatibility shims (older projects stored flows under a `scripts` key).

2.3 `Flow`: the graph

`Flow` (`ryvencore/Flow.py`) manages nodes and edges. A flow is a directed multigraph where the vertices are *ports*, not nodes. Three data structures matter:

```
self.nodes:            List[Node]
self.graph_adj:        Dict[NodeOutput, List[NodeInput]]   # out -> fan-out
self.graph_adj_rev:    Dict[NodeInput, Optional[NodeOutput]] # in -> 0 or 1 source
self.node_successors:  Dict[Node, List[Node]]              # for executors
```

The cardinality is asymmetric by design: an output may feed any number of inputs, but an input has *at most one* incoming connection. `connected_output(inp)` therefore returns a single port or `None`, which is why node code can treat "my input's value" as a scalar lookup.

`create_node(cls, data=None)` instantiates the class with the tuple `(flow, session)` — this is the mysterious `params` argument every node's `__init__` forwards to `super()`. It then wires port-change events, calls `node.initialize()` (which builds ports from `init_inputs` / `init_outputs`, or from saved data when loading), optionally calls `node.load(data)`, and finally places the node (`place_event()` fires).

`connect_nodes(out, inp)` validates the request (no self-connections, matching port types, correct directions), updates the adjacency maps, and then calls `executor.conn_added(out, inp, silent=...)`. In the default executor, **a new connection immediately triggers an update on the receiving node** — unless `silent=True`, which is exactly what project loading uses to avoid an update storm while rebuilding a saved graph.

2.4 `Node` and its ports

`Node` (`ryvencore/Node.py`) is the base for all node blueprints. Class attributes describe the blueprint (`title`, `tags`, `version`, `init_inputs`, `init_outputs`, `identifier`); instances hold live ports and state. The three methods that make up the *data-plane API* all delegate to the flow's executor:

```python
def update(self, inp=-1):            # request own update_event
    self.updating.emit(inp)
    self.flow.executor.update_node(self, inp)

def input(self, index) -> Optional[Data]:
    return self.flow.executor.input(self, index)

def set_output_val(self, index, data):
    assert isinstance(data, Data)    # payloads MUST be wrapped
    self.flow.executor.set_output_val(self, index, data)
```

That `assert` is why every Visual-SCF node wraps results in `Data(...)` (or a subclass) before emitting.

The *virtual methods* a node implementation may override form its life cycle:

Method	Called when	Typical use
`update_event(inp)`	an input fired, or `update()`	the node's computation
`place_event()`	node added to flow (before edges)	timers, addon access
`remove_event()`	node removed	stop threads/timers
`get_state()`	saving	return JSON-able dict
`set_state(d, v)`	loading (before edges)	restore fields
`rebuilt()`	loading, *after* edges exist	re-fire computation

Ports are thin: `NodeOutput` holds `val` (the current `Data`), `NodeInput` holds an optional `default`. There is no queueing and no copying — an input "having a value" literally means *the output it is connected to has a non-`None` `val`*.

2.5 `Data`: the payload envelope

`Data` (`ryvencore/Data.py`) wraps whatever a node emits in an object with a `payload` property plus serialization support (pickle by default). Two properties of the design are worth internalizing:

- **Payloads are shared, not copied.** If one output feeds two consumers and the first mutates the payload in place, the second sees the mutation. Subclasses can override the `payload` property to return copies if isolation matters.
- **Custom subclasses must be registered.** On load, output values are reconstructed by looking up the data-type identifier in `session.data_types`; unregistered types are skipped with an error. Ryven's `export_nodes(..., data_types=[...])` handles registration for package types like `MatrixData`.

> **Visual-SCF note**
>
> Visual-SCF relies on the sharing behavior implicitly: a PySCF `Mole` object flows by reference from `MolNode` to every consumer. Nothing mutates it downstream, so this is safe — but it is a convention, not something the framework enforces. The `hasattr(self.input(i), 'payload')` guard used throughout `inputs_ready()` works because `Node.input()` returns `None` when the connected output has never been set (and `None` has no `payload` attribute).

3 Flow execution: the executors

Execution semantics are pluggable. `Flow` holds an executor object, selected by the flow's *algorithm mode* (persisted in the project file); `Node.update()`, `Node.input()`, `Node.set_output_val()` and `Node.exec_output()` are all forwarded to it. Three algorithms exist in `ryvencore/FlowExecutor.py`:

3.1 `DataFlowNaive` (the default)

The default mode (`FlowAlg.DATA`) is a synchronous push model. Setting an output propagates *immediately and recursively*:

```
# Node.set_output_val() =>
def set_output_val(self, node, index, data):
    out = node.outputs[index]
    out.val = data
    for inp in self.graph[out]:
        inp.node.update(inp=inp.node.inputs.index(inp))
```

So a call to `set_output_val` inside node A's `update_event` runs node B's `update_event` *before A's has returned* — flow execution is plain Python call-stack recursion, on the Qt main thread, with no scheduling, no deferral, and no cycle detection. Consequences:

- **Order:** with multiple outgoing edges, activation order is the list order of `graph_adj[out]` (i.e. connection creation order), but is documented as undefined — do not rely on it.
- **Diamonds are expensive:** if A feeds B and C, which both feed D, then one update of A updates D

twice; nested diamonds grow exponentially.

- **Cycles do not terminate:** a feedback edge means infinite recursion until Python's recursion limit kills it. The engine *assumes* no non-terminating feedback loops; breaking a cycle is the node author's job.
- **Exceptions are contained:** `update_node` wraps `update_event` in `try/except`; failures go to `node.update_err(e)`, which emits the `update_error` event (shown by the GUI as a red indicator bubble on the node item) rather than unwinding the whole propagation.
- **New connections fire immediately:** `conn_added` updates the receiving node right away (except during silent/loading operations). A node must therefore tolerate being updated when only some of its inputs are connected — this is the entire reason for the `inputs_ready()` idiom.

Visual-SCF note

The SCF feedback loop (`SCFStepNode`) exists precisely because of the naive executor's cycle behavior. The graph `SCF Step` → `Fock` → `Get MO Coeff` → `Make 1-RDM` → `SCF Step` is a true cycle; if `SCFStepNode.update_event` propagated on arrival of "New 1-RDM", `set_output_val` would recurse forever. Instead the node absorbs that input silently and only re-emits when the user clicks *Step* — turning the runaway recursion into one user-paced lap around the loop per click. Each click still executes the whole chain synchronously before returning to the Qt event loop.

3.2 `DataFlowOptimized`

An alternative mode (`FlowAlg.DATA_OPT`) that guarantees each connection activates *at most once per execution*. When an execution starts (some node or output is updated from outside), it runs a BFS/DP pass over the successor component to count, for every node, how many incoming connections could still deliver data (`generate_waiting_count`). Outputs written during the execution are buffered (`out.val` is set, but successors are not updated); a node's outputs are propagated only when its waiting count reaches zero, i.e. when no predecessor can still change its inputs. The DP table is cached between executions while the graph is unchanged (`flow_changed` flag maintained by `Flow`).

This fixes the diamond blow-up, but tightens the assumptions: the graph must be acyclic and nodes must not modify their ports during execution. It would *not* be safe for Visual-SCF's cyclic SCF graph.

3.3 `ExecFlowNaive`

The third mode (`FlowAlg.EXEC`) implements UnrealEngine-blueprint-style execution with dedicated `exec`-type ports: triggers travel forward along exec connections (`exec_output()`), while data is *pulled backward on demand* — when an updating node calls `self.input(i)` and the connected predecessor has not yet updated in this execution, the predecessor's `update_event(-1)` is invoked first so it can push a fresh value. Ryven exposes the mode per flow, but Visual-SCF uses pure data flows only.

4 The nodes package system

4.1 Anatomy and import mechanics

A nodes package is a directory containing a `nodes.py` (mandatory) and usually a `gui.py`. It is identified by its *directory basename* — which is why the project's package is called `pyscf` and why that name collides gracefully with the `pyscf` library only because Ryven loads the file by path, not via `import`.

Loading happens in `ryven/main/packages/nodes_package.py::import_nodes_package()`:

1. The environment variable `RYVEN_MODE` must be set (`'gui'` or `'no-gui'`); the editor sets it in `Ryven.py` before any package import.
2. `NodesEnvRegistry.current_package` is set to the package being imported — global state consumed by `export_nodes()`.
3. `nodes.py` is executed via `load_from_file()` (`importlib` machinery, no package installation involved).

4. If in GUI mode, all functions registered with `@on_gui_load` are invoked (this is when `gui.py` gets imported); in headless mode they are skipped entirely, so `nodes.py` must never import Qt at module level.
5. The registry's accumulated node and data classes are consumed and handed to the session for registration.

4.2 What `export_nodes()` actually does

Beyond collecting classes in the registry, `export_nodes()` (`ryven/main/packages/node_env.py`) performs identifier surgery on each node class:

- builds the identifier as `<class name>` if not explicitly set, then prefixes it with the package name: the saved-file identifier of `MolNode` is `pyscf.MolNode`;
- appends fallbacks to `legacy_identifiers` so projects saved under older naming schemes still resolve;
- registers exported `Data` subclasses under `<pkg>.<ClassName>` the same way.

When a project is loaded, `node_from_identifier()` resolves each saved identifier against registered classes, falling back to `legacy_identifiers` — this is the entire backward-compatibility story for renamed/moved node classes.

4.3 Binding GUIs: `@node_gui`

`ryven.gui_env` provides the `node_gui(NodeClass)` decorator, which simply sets `NodeClass.GUI = <NodeGUI subclass>` (with a guard against double registration). Nothing is instantiated at binding time; the attribute is read later when a node item is built for a placed node. GUI classes are *inherited* by node subclasses unless overridden.

5 ryvencore-qt: how the GUI is wired

5.1 `SessionGUI` and `FlowView`

`SessionGUI` wraps the core session (created with `gui=True, load_addons=True`), stores itself as `session.gui`, and subscribes to `flow_created` — so whenever a flow appears (new or loaded), it constructs a matching `FlowView`. It also registers the frontend's `complete_data` function with ryvencore (see §6).

`FlowView` is a `QGraphicsView/QGraphicsScene` pair, and is where all direct user interaction lives: placing nodes (it listens to `flow.node_added` and creates a `NodeItem`), dragging connections between `PortItem`s (validated through `flow.check_connection_validity` before `flow.connect_nodes` is called), selection, undo/redo commands (`FlowCommands.py`), zooming, themes (`FlowTheme.py`), and the node-list popup for adding nodes.

5.2 `NodeGUI`: the mediator

For each placed node, the frontend builds a pair of objects:

- `NodeGUI` (`ryvencore_qt/src/flows/nodes/NodeGUI.py`) — a `QObject` constructed with `(node, session_gui)`. Its constructor does `setattr(node, 'gui', self)` — **this is where the `node.gui` attribute comes from**, and why headless nodes must guard with `hasattr(self, 'gui')`: in no-GUI mode the attribute simply never exists. It subscribes to the node's core events (`updating`, `update_error`, ports added/removed) and re-emits them as Qt signals. It also owns the `actions` dict — the node's right-click menu — mapping labels to `{'method': callable}` entries, (de)serialized by method *name*.
- `NodeItem` — the actual `QGraphicsItem` in the scene: title label, port items, the collapsible body, the error indicator, and the animator. It reads the class attributes of the bound `NodeGUI` subclass: `color`, `style`, `display_title`, `main_widget_class` and `main_widget_pos` (`'below ports'` or `'between ports'`).

If `main_widget_class` is set, `NodeItem` instantiates it, passing `(node, item, node_gui)` — the widget's `params`. The widget is a normal `QWidget` embedded into the graphics scene through a `QGraphicsProxyWidget` subclass (`FlowViewProxyWidget`). Embedding via proxy is exactly why Visual-SCF's 3D viewer had to become a pure-QPainter widget: native surfaces (GL/VTK) do not survive inside `QGraphicsProxyWidget`.

Main widgets participate in persistence through their own `get_state()/set_state()` pair: `NodeItem` pickles the

widget state into the item's data under `'main widget data'` on save and restores it on load. Note the asymmetry with nodes: widget `set_state(data)` takes no version argument.

5.3 Update flow, end to end

Putting sections 3 and this one together, a click on Visual-SCF's *Step* button travels like this:

1. Qt delivers the click to `SCFStepWidget`; the handler calls `self.node.step()` — widget-to-node communication is a plain method call.
2. `step()` calls `set_output_val(0, Data(...))`, entering the naive executor, which synchronously updates every connected node (`FockNode.update_event`, then its consumers, ...).
3. Each updated node that wants to refresh its view calls `self.gui.<method>(...)`; the `NodeGUI` forwards to the embedded main widget. (The framework itself also emits `updating`, which the item uses for its little update animation.)
4. The whole cascade completes before the click handler returns; the GUI repaints afterwards. Long-running node computations therefore freeze the editor — there is no background execution in the framework.

6 Serialization and project loading

6.1 What a project file contains

`Session.serialize()` produces the JSON structure saved as e.g. `testsave.json`:

```
{ "flows": { "<title>": {
      "algorithm mode": "data",
      "nodes":       [ <node dicts> ],
      "connections": [ {parent node index, output port index,
                        connected node, connected input port index} ],
      "output data": [ {data: <pickled Data>,
                        dependent node outputs: [n_i, o_i, ...]} ]
  }},
  "addons": { ... } }
```

Per node, `Node.data()` stores the identifier, version, and port lists, plus — crucially — the user state under the `'state data'` key as `serialize(self.get_state())`, where `serialize` is `base64(pickle(...))`. Two practical implications: whatever `get_state()` returns must be *picklable*, and project files are not hand-editable where state is concerned.

Output values are serialized once per `Data` *object*, with a list of (node, output) index pairs that shared it — object identity is preserved across save/load for fan-out values.

6.2 GUI state piggybacks via `complete_data`

The core dicts contain no coordinates, no widget state. When saving, `Base.complete_data(data)` invokes the function registered by `SessionGUI`; it recursively walks the produced dict, finds components by their `GID`, and lets each corresponding GUI object merge its own state in — `FlowView` adds node positions and drawings, `NodeItem` adds `'main widget data'` and the `NodeGUI` actions/display-title. The result: one file, two layers of ownership, and a core that stays GUI-free.

6.3 The load sequence (and why `rebuilt()` exists)

`Flow.load_components()` restores a graph in a strict order:

1. **Create nodes.** Each node class is resolved by identifier; `Node.load()` rebuilds ports *from saved data* (not from `init_inputs`) and calls `set_state(deserialize(state), version)` inside a `try/except` — a crashing `set_state` prints an error and leaves the node default-initialized instead of aborting the load.

2. **Restore output values** from the deduplicated `output data` section.
3. **Reconnect edges** — with `silent=True`, so `conn_added` does *not* trigger updates; the graph reassembles quietly.
4. **Call `rebuilt()` on every node.** Only now are connections guaranteed to exist, which is why re-firing computation belongs here and not in `set_state`. Order across nodes follows the saved node list, so upstream data may or may not be present when a given node's `rebuilt()` runs.

Visual-SCF note

This explains three Visual-SCF conventions at once. (1) `set_state` must default-load missing keys: old saves simply lack newer fields, and an exception here silently resets the node. (2) `MolNode.rebuilt()` re-calls `build()` so a loaded project reanimates from the roots — downstream nodes then receive ordinary updates through reconnected edges. (3) `CASSCFNode` cannot assume its inputs are live during `rebuilt()` (rebuild order is node-list order, and upstream recomputation may not have happened yet), hence its `_rerun_on_ready` deferral to the next `update_event`.

7 Add-ons

Add-ons (`ryvencore/AddOn.py`, instances in `ryvencore/addons/`) are session-level services with lifecycle hooks: `register(session)`, `connect_flow_events(flow)`, `on_node_removed(node)`, plus `extend_node_data(node, dict)` which lets an add-on append to every node's serialized dict, and their own `data()/load()` for project persistence. Nodes reach them with `self.get_addon(name)`.

Two ship with ryvencore:

- **Variables** (`addons/Variables.py`) — named, per-flow variables with a subscription API: `subscribe(node, name, callback)` invokes the callback whenever the variable is set. The editor shows them in a list widget per flow. Visual-SCF's (currently disabled) `EditMatrixNode` uses this to re-evaluate matrices when referenced variables change.
- **Logging** — per-node loggers surfaced in the editor's log widgets.

8 Editor startup, end to end

What happens when `scripts/run.sh` executes `ryven -q pyside6 -n .../pyscf -n .../plotting2 -n .../matrices`:

1. `ryven.main.Ryven.run()` parses CLI args into a `Config` (`main/config.py`). *A local fork patch lives here:* upstream declared the `nodes` default as `set()`, the fork changes it to `list()` so multiple `-n` packages accumulate in order instead of being deduplicated into an unordered set.
2. It verifies the requested Qt binding, sets `RYVEN_MODE=gui` and `QT_API=pyside6`, and only then imports GUI modules (all Qt imports in the stack go through `qtpy`, which reads `QT_API`).
3. The `QApplication` is created; fonts are registered; the startup dialog appears unless a project was passed (the dialog also lets you pick packages and projects interactively).
4. Package paths are resolved into `NodesPackage` objects; if a project file was given, the packages it requires are checked for availability (projects record their required packages, and loading refuses to proceed with missing ones).
5. `MainWindow` builds the `SessionGUI`, imports each nodes package (running `nodes.py`, then `gui.py` via the `on_gui_load` hooks), registers the exported node and data types with the core session, and — if a project was given — calls `session.load()`, which cascades into the flow/node loading sequence of §6.
6. `app.exec_()` enters the Qt event loop. Unless `--verbose` is given, stdout/stderr are redirected into the in-app console — and the fork patches this in two ways: `RedirectOutput` (`gui/main_console.py`) implements the file-like interface (`flush/isatty/writelines`) that PySCF's logger requires, and the redirection is installed *before* `MainWindow` is built, so objects that capture `sys.stdout` at construction time (PySCF `Mole`s created during project load) also log to the console.

9 Cheat sheet: framework contracts Visual-SCF relies on

Framework behavior	Visual-SCF convention it induces
`conn_added` updates the target node immediately; inputs may be partially connected	guard every `update_event` with `inputs_ready()` using `hasattr(port, 'payload')`
`set_output_val` asserts `isinstance(data, Data)`	wrap all payloads: `Data`, `MolData`, `MatrixData`
naive executor recurses synchronously; cycles never terminate	`SCFStepNode` swallows feedback-edge updates; Step/Reset buttons drive the loop
`node.gui` is set only in GUI mode, by `NodeGUI.__init__`	all view updates behind `hasattr(self, 'gui')`; `nodes.py` never imports Qt
`set_state` runs before edges exist; exceptions reset the node silently	`data.get(key, default)` everywhere; recomputation deferred to `rebuilt()`
`get_state` is pickled into the save file	keep state JSON-simple and picklable; schema changes need back-compat defaults (test against `testsave.json`)
main widgets live inside `QGraphicsProxyWidget`	no native GL surfaces; the orbital viewer is pure QPainter (`plotting2/qpainter3d.py`)

Where to look things up:

- Engine — `~/Projects/ryvencore/ryvencore/`:
  `Flow.py` (graph + module docstring on execution modes), `FlowExecutor.py`, `Node.py`, `Session.py`, `Data.py`, `Base.py`, `addons/`
- Qt layer — `~/Projects/Ryven/ryvencore-qt/ryvencore_qt/src/`:
  `SessionGUI.py`, `flows/FlowView.py`, `flows/nodes/NodeGUI.py`, `flows/nodes/NodeItem.py`, `flows/nodes/WidgetBaseClasses.py`
- Editor & package system — `~/Projects/Ryven/ryven-editor/ryven/`:
  `main/Ryven.py`, `main/config.py`, `main/packages/nodes_package.py`, `main/packages/node_env.py`, `main/packages/gui_env.py`, `example_nodes/`